Google Skills

The solution:

Built-in tools

Production-ready capabilities

"These built-in tools provide ready-to-use functionality such as Google Search or code executors that provide agents with common capabilities."

Available built-in tools:

- **Google Search:** Web search with grounding (this part)
- **Code execution:** Execute Python code safely (this part)
- **Vertex AI search:** Search enterprise documents
- **Vertex AI RAG engine:** Retrieval-augmented generation
- **BigQuery:** Query data warehouses
- **Spanner:** Query Spanner databases
- **Bigtable:** Query Bigtable databases

Reference: ADK Docs - Built-in Tools

For this introductory lesson, we focus on:

1. Google Search:
   Most universally useful for current information
2. Code execution:
   Enables mathematical and computational tasks

Built-in tools overview:

```
Python
graph LR
    A[Built-in Tools] --> B[Google
Search]
    A --> C[Code Execution]
    A --> D[Vertex AI Search]
    A --> E[Vertex AI RAG]
    A --> F[BigQuery]
    A --> G[Spanner]
    A --> H[Bigtable]

    B --> I[Real-time web info]
    C --> J[Python execution]
    D --> K[Enterprise docs]
    E --> L[Document retrieval]
    F --> M[Data warehouses]
    G --> N[Spanner databases]
    H --> O[Bigtable data]

    style B fill:#4285f4,color:#fff
    style C fill:#4285f4,color:#fff
    style D fill:#f0f0f0
    style E fill:#f0f0f0
    style F fill:#f0f0f0
    style G fill:#f0f0f0
    style H fill:#f0f0f0
```

Built-in tools available in ADK (blue = covered in this part)

Core concepts

1. Google Search tool

From ADK documentation:

"The `google_search` tool allows the agent to perform web searches using Google Search. The `google_search` tool is only compatible with Gemini 2 models."

What makes this a tool

- Real-time web search results
- Authoritative, up-to-date information
- Automatic grounding of responses in sources
- Search suggestions (must be displayed per policy)

How to use:

```python
from google.adk.agents import LlmAgent
from google.adk.tools import google_search  # Import built-in tool

search_agent = LlmAgent(
    model='gemini-2.5-flash',  # Must use Gemini 2.0 or later
    name='search_agent',
    instruction="You help users find information using web search.",
    tools=[google_search]  # Add Google Search tool
)
```

Key differences from part 1:

- **Import instead of define:**
  From `google.adk.tools import google_search`
- **No function writing:**
  ADK provides the complete implementation
- **Production-ready:**
  Optimized and maintained by Google
- **Model requirement:**
  Only works with Gemini 2.0+ models

What happens when the agent uses this tool:

1. Agent determines search is needed based on user query
2. Agent generates search query automatically
3. Google Search returns relevant results
4. Agent synthesizes results into response
5. Response includes source citations

Reference: ADK Docs - Google Search

Grounding with Google Search

What is grounding?

Grounding connects agent responses to authoritative sources, reducing hallucinations and improving accuracy. Instead of relying solely on the LLM's training data, the agent bases answers on current, verifiable information.

Benefits of Google Search grounding:

- Responses based on real-time web data
- Sources cited for verification
- Reduced hallucinations
- Up-to-date information beyond training cutoff

IMPORTANT policy requirement:

When using Google Search grounding, you **MUST** display search suggestions (`renderedContent`) in your application UI. This is mandatory per Google Search usage policy.

```
None
# In your application code (not
agent.py)
response = runner.run(...)
if hasattr(response,
'rendered_content') and
response.rendered_content:

display_html(response.rendered_content)
# REQUIRED by policy
```

Why this matters:

Grounding responses include HTML search suggestions that must be shown to users for policy compliance. Non-display violates usage terms.

Reference: ADK Docs - Google Search Grounding

2. Code execution tool

From ADK documentation:

"The built_in_code_execution tool enables the agent to execute code, specifically when using Gemini 2 models. This allows the model to perform tasks like calculations, data manipulation, or running small scripts."

What it provides:

- Execute Python code safely
- Perform precise calculations
- Process and analyze data
- Run algorithms and transformations

How to use:

```python
from google.adk.agents import LlmAgent
from google.adk.code_executors import BuiltInCodeExecutor

code_agent = LlmAgent(
    model='gemini-2.5-flash',  # Must use Gemini 2.0 or later
    name='code_agent',
    instruction="You help users with calculations and data processing.",
    code_executor=BuiltInCodeExecutor()  # Enable code execution
)
```

Note the difference: Code execution uses code_executor parameter, not tools list.

What the agent can do with code execution:

- Perform complex mathematical calculations
- Process and transform data
- Generate visualizations
- Implement algorithms
- Validate computations

Key advantages:

- **Precise calculations:** No rounding errors from LLM estimation
- **Complex operations:** Can run multi-step algorithms
- **Verification:** Code can be inspected and verified
- **Reproducible:** Same code produces same results

Example scenario:

```
User: "Calculate the compound interest on $10,000 at 5% annual rate for 10 years"

Agent thinking:
1. Recognizes this needs precise calculation
2. Generates Python code:
   principal = 10000
   rate = 0.05
   time = 10
   amount = principal * (1 + rate) ** time
   interest = amount - principal
3. Executes the code
4. Returns: "The compound interest is $6,288.95. The total amount is $16,288.95."
```

Reference: ADK Docs - Code Execution

3. Built-in tools and session state (preview)

Connection to lesson 4: Managing state and memory

Built-in tools can work with session state for personalized, context-aware behavior. While full integration is covered in future lessons, here's what's possible:

Use cases:

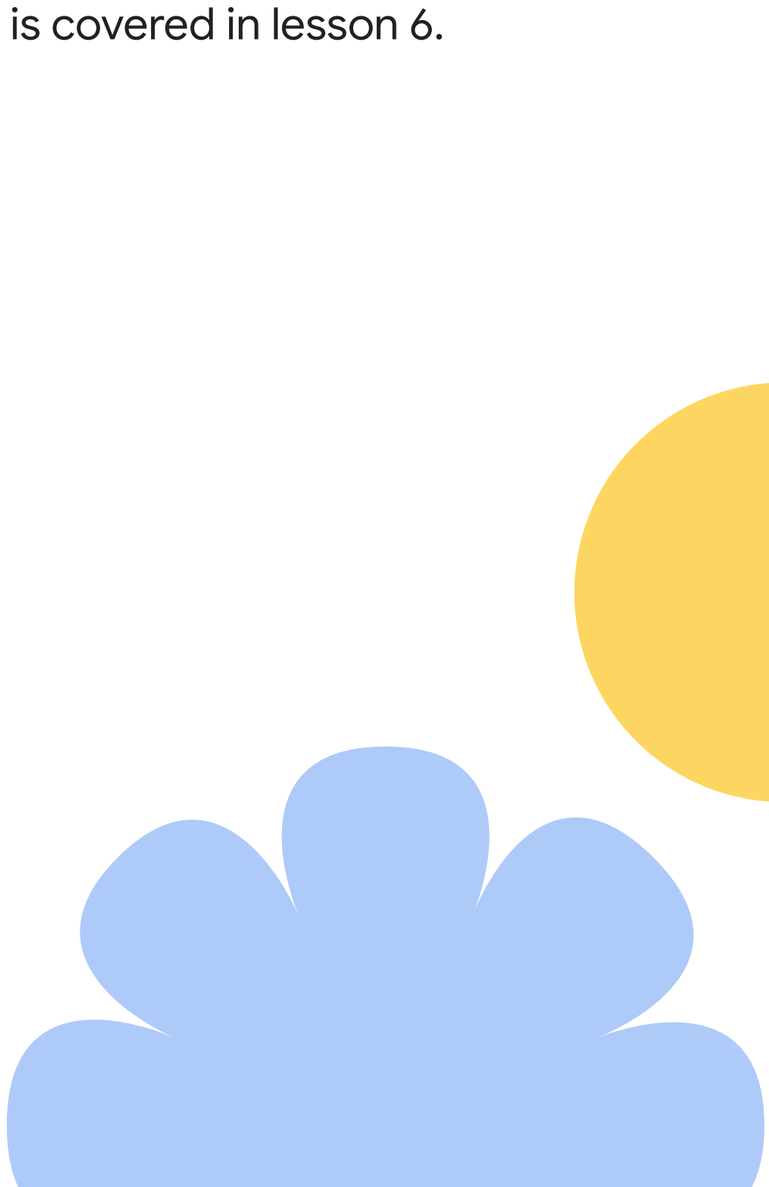
- Google Search can use user preferences from state (e.g., preferred language, location)
- Code Execution results can be saved to state for later reference
- Search queries can be personalized based on `user:` namespace data

Example pattern (covered in lesson 6):

```python
# User preference in state
state["user:preferred_units"] = "metric"

# Agent instruction can use templating
instruction = """
When performing calculations, use {user:preferred_units?imperial} units.
Use code execution for precise calculations.
"""
```

Preview: Full state integration with built-in tools, including accessing state within tool execution context, is covered in lesson 6.



4. When to use built-in tools

Comparison: Built-in vs custom function tools

Aspect	Built-in tools	Custom function tools
Setup	Import and use	Write function implementation
Maintenance	ADK team maintains	You maintain
Optimisation	Pre-optimized for LLMs	You optimize
Customization	Limited to tool's features	Full control
Examples	Google Search, code execution	`get_capital_city`, `calculate_shipping`
Best for	Common capabilities	Business-specific logic

Use built-in tools when you:

- Need common capabilities (search, code execution)
- Want production-ready, maintained solutions
- Don’t want to implement complex integrations
- Need optimization for LLM interaction
- Require enterprise-grade reliability

Use custom function tools when:

- You need business-specific logic
- Integrating with proprietary systems
- Implementing unique algorithms
- You need simple, specific tasks unique to your application
- You need full control over implementation

Example decision:

None

```
Task: "Add web search to agent"
Decision: Use google_search (built-in)
Why: Common capability, complex to implement, maintained by ADK

Task: "Look up inventory in our database"
Decision: Write custom function tool
Why: Proprietary system, simple database query, specific to your business
```

5. Limitations of built-in tools

From ADK documentation:

"Currently, for each root agent or single agent, only one built-in tool is supported.
No other tools of any type can be used in the same agent."

What this means:

❌ NOT SUPPORTED: Built-in tool + custom tool in same agent

```python
# This will NOT work
root_agent = LlmAgent(
    model='gemini-2.5-flash',
    tools=[google_search, my_custom_function]  # ❌ Can't mix
)
```

❌ NOT SUPPORTED: Multiple built-in tools in same agent

```python
# This will NOT work
root_agent = LlmAgent(
    model='gemini-2.5-flash',
    tools=[google_search],  # ❌ Can't have both
    code_executor=BuiltInCodeExecutor()  # ❌ Can't have both
)
```

Current limitation:

Only one built-in tool per agent. Multi-agent solutions (using specialized agents for different tools) are covered in future lessons on multi-agent coordination.

Python workaround (advanced):

ADK Python has a built-in workaround for `GoogleSearchTool` and `VertexAiSearchTool` using `bypass_multi_tools_limit=True`. This is beyond the scope of this introductory lesson.

Reference: ADK Docs - Built-in Tools Limitations

Hands-on example

Building agents with built-in tools

We'll build two separate agents to demonstrate both tools:

1. **Research assistant**
   Uses Google Search for current information
2. **Math assistant**
   Uses Code Execution for calculations

Example 1: Research assistant with Google Search

What you'll build: A research assistant that answers questions using Google Search grounding.

Step 1: Create the project

```shell
adk create research_assistant
cd research_assistant
```

Step 2: Write the agent

Replace the contents of `agent.py` with:

```python
"""
Research Assistant Agent
Demonstrates ADK's Google Search built-in tool for real-time information.

Reference: https://google.github.io/adk-docs/tools/built-in-tools#google-search
"""

from google.adk.agents import LlmAgent
from google.adk.tools import google_search  # Import Google Search tool

# Create research assistant with Google Search
root_agent = LlmAgent(
    model='gemini-2.5-flash',  # Must use Gemini 2.0+ for google_search
    name='research_assistant',
    description='Helps users research topics using Google Search.',
    instruction="""
    You are a research assistant that helps users find accurate, up-to-date
information.
```

```
Your approach:
    1. When users ask questions requiring current information, use Google Search
    2. Base your answers on the search results
    3. Cite sources when providing information
    4. If search results are insufficient, acknowledge limitations

    Always prioritize accuracy over speculation. If you're unsure, say so.
    """,
    tools=[google_search]  # Enable Google Search grounding
)
```

Code walkthrough

Line 7: Import Google Search

- Import the built-in tool from `google.adk.tools`
- No implementation needed - ADK provides it

Lines 10-24: Create agent with Google Search

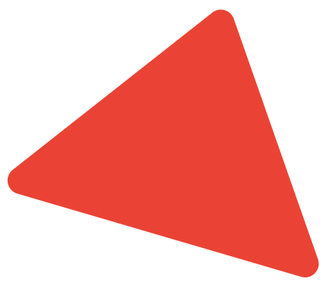
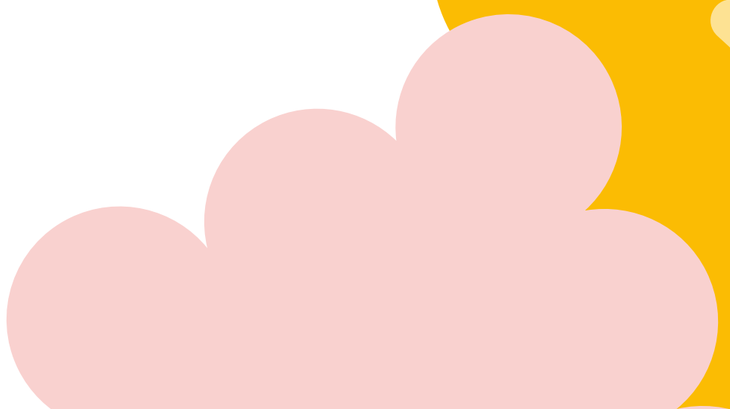
- `model='gemini-2.5-flash'` - Gemini 2.0+ required
- `tools=[google_search]` - Add the built-in tool
- Instruction guides the agent to use search for current information

Step 3: Run and test

Shell

```
adk web
```

Navigate to `http://localhost:8000` in your browser.


Step 4: Test scenarios

Test 1: Simple calculation

You: `What are the latest developments in renewable energy?`

Expected behavior:

- Agent uses Google Search to find current information
- Response includes recent developments
- Sources are cited
- Information is up-to-date

What to notice:

- Agent automatically searched the web
- Response is grounded in current data, not training data
- Sources provide verifiability

Test 2: Factual lookup

You: `Who is the current CEO of Microsoft?`

Expected behavior:

- Agent searches for current information
- Returns accurate, up-to-date answer
- Cites source

What to notice:

- Answer reflects current reality, not training data
- Search provides authoritative sources

Test 3: Complex research

You: `Compare electric vehicles and hydrogen fuel cell vehicles`

Expected behavior:

- Agent searches for multiple perspectives
- Synthesizes information from search results
- Provides balanced overview with sources

What to notice:

- Agent can handle complex, multi-faceted queries
- Search results inform comprehensive response

IMPORTANT: Search suggestions policy

If your response includes search suggestions (in `renderedContent`), you **MUST** display them in your application UI. This is a mandatory policy requirement.

For production applications, handle it like this:

```python
# In your application code (not agent.py)
from google.adk.runners import Runner

runner = Runner(...)
response = runner.run(...)

# Check for and display search suggestions
if hasattr(response, 'rendered_content') and response.rendered_content:
    # Display the HTML search suggestions in your UI
    # This is REQUIRED by Google Search grounding policy
    display_in_ui(response.rendered_content)
```

Example 2: Math assistant with code execution

What you'll build: A math assistant that performs precise calculations using code execution.

Step 1: Create the project

```shell
adk create math_assistant
cd math_assistant
```

Step 2: Write the agent

Replace the contents of `agent.py` with:

```python
"""
Math Assistant Agent
Demonstrates ADK's Code Execution built-in tool for calculations.

Reference: https://google.github.io/adk-docs/tools/built-in-tools#code-execution
"""

from google.adk.agents import LlmAgent
from google.adk.code_executors import BuiltInCodeExecutor  # Import code executor
```

```
# Create math assistant with code execution
root_agent = LlmAgent(
    model='gemini-2.5-flash',  # Must use Gemini 2.0+ for code execution
    name='math_assistant',
    description='Helps users with mathematical calculations and analysis.',
    instruction="""
    You are a math assistant that helps users with calculations and mathematical
analysis.

    Your capabilities:
    1. When users ask for calculations, use code execution for precision
    2. Show your work by explaining the calculation steps
    3. Verify results by running the code
    4. Handle complex mathematical operations (statistics, algebra, etc.)

    Always use code execution for numerical calculations to ensure accuracy.
    """,
    code_executor=BuiltInCodeExecutor()  # Enable code execution
)
```

Code walkthrough

Line 8: Import code executor

- Import from google.adk.code_executors
- Provides safe Python code execution

Lines 11–25: Create agent with code execution

- code_executor=BuiltInCodeExecutor() - Enable code execution (note: not in tools list)
- Instruction guides agent to use code for calculations

Step 3: Run and test

Shell

```
adk web
```

Navigate to http://localhost:8000 in your browser.

Step 4: Test scenarios

Test 1: Simple calculation

You: `Calculate 15% tip on a $87.50 bill`

Expected behavior:

- Agent generates Python code to calculate
- Code executes: 87.50 * 0.15
- Returns precise result: $13.13

What to notice:

- Calculation is precise (no rounding errors)
- Agent shows or explains the calculation

Test 2: Complex calculation

You: `What's the compound interest on $5,000 invested at 6% annual rate for 8 years, compounded monthly?`

Expected behavior:

- Agent generates compound interest formula in Python
- Code executes the calculation
- Returns precise result with explanation

What to notice:

- Complex financial formula handled correctly
- Code execution ensures mathematical accuracy

Test 3: Data processing

You: `Calculate the average, median, and standard deviation of these numbers: 12, 15, 18, 20, 22, 25, 28, 30`

Expected behavior:

- Agent generates Python code using statistical functions
- Code calculates all three metrics
- Returns accurate results

What to notice:

- Can perform multiple calculations in one request
- Statistical operations handled precisely

Key takeaways

Built-in tools:

- **Ready-to-use capabilities** imported from `google.adk.tools` or `google.adk.code_executors`
- **Production-ready:** Maintained and optimized by the ADK team
- **Two tools in this part:** Google Search (for information) and Code Execution (for calculations)

Google search tool:

- **Real-time web search** with automatic grounding
- **Requires Gemini 2.0+** models
- **Policy requirement:** Must display search suggestions when provided
- **Import:** `from google.adk.tools import google_search`
- **Usage:** `tools=[google_search]`

Code execution tool:

- **Execute Python code** for precise calculations
- **Requires Gemini 2.0+** models
- **Safe execution** in controlled environment
- **Import:** from `google.adk.code_executors import BuiltInCodeExecutor`
- **Usage:** `code_executor=BuiltInCodeExecutor()` (note: not in tools list)

Aspect	Built-in tools	Custom function tools
Implementation	Import and use	Write function
Maintenance	ADK maintains	You maintain
Use for	Common capabilities	Business logic

Current limitations:

- **One built-in tool per agent** - Cannot mix built-in tools or combine with custom tools in same agent
- **Solution:** Use sub-agents with AgentTool (covered in future lessons)
- **Workaround exists** for some tools in Python (advanced topic)

When to use:

- **Built-in tools:** Common capabilities (search, code execution, database queries)
- **Custom tools:** Business-specific logic, proprietary systems